DataBackup

Linux 目录树备份与还原软件

《软件开发综合实验》课程项目

组长：王科瑾（单人组）

电子科技大学 · 2026 年 11 月

v1.0.1

需求范围：基础功能与全部扩展项

课程要求分为基础 40 分与扩展项；本项目 18 条功能需求全部实现，未做实时备份与网络备份。

需求	内容	计分
FR-01 / 02	把目录树备份为单个文件；还原到指定目录	基础 40
FR-03 / 04	8 种文件类型（含硬链接、设备、套接字）；权限、属主、纳秒时间	扩展 20
FR-05 ~ 10	路径、类型、名字、时间、尺寸、用户组六个过滤维度，可组合	扩展 25
FR-11	自研 MyPack 归档格式（512B 头记录，仿 tar）	扩展 10
FR-12	自研 Huffman 压缩（1 MiB 分块流式）	扩展 10
FR-13	自研 AES-256 加密： PBKDF2 派生 + CBC + HMAC	扩展 10
FR-14 / 15	备份列表、详情、删除；仓库路径设置	支撑
FR-16 ~ 18	Electron 客户端；错误口令与路径穿越防护； JSONL 进度	扩展 10

明确不做

实时备份、网络备份。

两项都需要常驻服务或服务器，与 " 单机、单文件、可离线 " 的课程场景不匹配。

可选项（已完成）

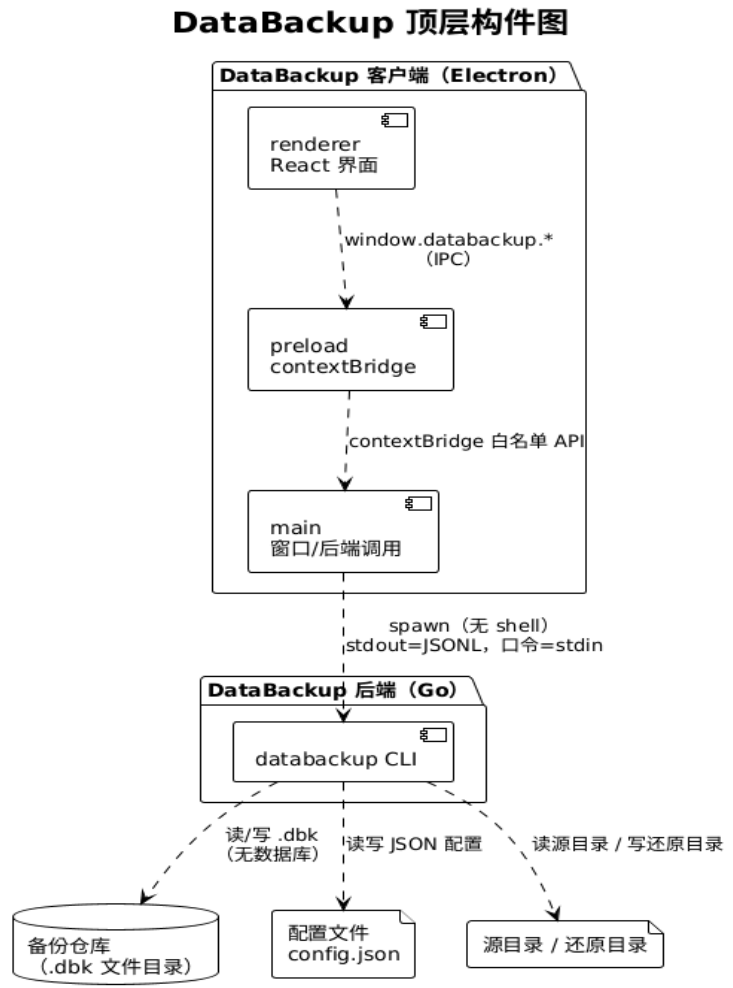
Windows 便携包：同一套后端交叉编译，平台不支持的元数据静默跳过，仅用于课堂演示兜底。

计分口径见 docs/project/course-requirements.md ；扩展项按课程 85 分上限核算。

架构：进程边界清晰，客户端不含业务逻辑

- **编译型后端** Go 1.27，生产代码零第三方模块；自研约束由 scripts/check-imports.sh 强制。
- **唯一接口** 客户端 spawn CLI， stdout 输出 JSON Lines；协议冻结在 cli-contract.md 。
- **无数据库** .dbk 头尾记录元信息， list / info 只读文件首尾，不需要口令。
- **口令隔离** 口令经 stdin 传入，不进参数、环境变量与日志。

契约双端一致： Go 类型与 TypeScript 类型由测试比对同一组 JSON 样例，字段漂移会直接失败。



图：顶层构件（ docs/deliverables/diagrams/component-top.puml ）

备份流水线：写临时文件，校验通过后才发布



环节	关键做法	失败时的行为
scan	lstat 遍历、稳定排序、硬链接按 inode 去重	不可读项跳过并计数，不中断任务
pack	头记录定长 512 B，长路径溢出到扩展块	路径不合规返回 ErrUnsafePath
compress	每块独立码表，内存与输入大小无关	随机数据不膨胀，最差 1.0
encrypt	encrypt-then-MAC，尾部记录统计与校验	篡改或口令错误返回认证失败（退出码 3）
publish	先写 .dbk.partial， fsync 后硬链接发布	同名已存在返回 E_EXISTS；取消删除半成品

还原方向相反：先对整文件做 MAC/SHA 校验，通过后才开始写目标目录，避免 " 还原到一半发现备份已损坏 " 。

MyPack： 512 字节定长头记录 + 全类型支持

自研格式

魔数 / 版本 / 类型 / 标志	CRC-32	长度与元数据字段	时间戳（秒 + 纳秒）	名字区 408 B

小端二进制；字段定长，读取时逐项校验；名字放不下时用扩展块续写（每块 512 B ），归档以两个全零块结尾。

类型码	对象	数据
0	普通文件	size 字节，补齐到 512 的倍数
1 / 2	硬链接 / 软链接	硬链接存目标路径；软链接原样保存
3 / 4	字符设备 / 块设备	设备主次号写入头记录
5 / 6 / 7	目录 / 命名管道 / 套接字	无数据，仅元信息

往返一致性由 golden 文件与 testdata 目录树断言：类型、权限、 uid/gid 、 atime/mtime（含纳秒）逐项比对。

读取端的安全校验

- 拒绝绝对路径、空段、 `.` 与 `..` 段、 NUL 字节。
- 拒绝以已出现软链接为祖先的后续路径，堵住借软链接写出还原目录的攻击。
- CRC-32 、保留位、纳秒范围、扩展块个数与结尾零块逐项校验；出错后持续报同一错误。

4,881,324

次 FuzzReader 执行（ 30 s ），无 panic 、无死循环。

Huffman： 分块流式编码，内存不随文件增长



每个 1 MiB 块独立建树，码长表写进块头，解码端按规范码重建码表——所以可以逐块流式读写，不需要把整个文件读进内存。

实测（512 MiB 文本， WSL2 + NVMe）

场景	耗时	吞吐	输出大小	压缩率
备份，无压缩	3.22 s	≈167 MB/s	536.9 MB	1.000
备份， Huffman	5.51 s	≈97 MB/s	330.8 MB	0.616
还原， Huffman	5.67 s	≈95 MB/s	—	—
备份，随机数据	7.76 s	≈69 MB/s	537.0 MB	1.000

10,000 个小文件：备份 0.16 s 、还原 0.36 s ；每文件摊薄开销约 16 µs （含 lstat 、头记录与元数据）。

边界条件

- 空输入、单字节、全同字节、跨块大文件都有用例。
- 随机数据不膨胀：压缩率不会超过 1.0 。
- 码长表非法或码表不完整时解码直接报错，不输出半个文件。
- 单线程实现是吞吐上限的主要原因，见第 12 页。

加密：口令派生双密钥，先加密后认证



密钥派生（RFC 8018 §5.2）

```
K = PBKDF2-HMAC-SHA256( 口令 , salt, 100 000 轮 , 64 B)

K[0:32] → AES-256 密钥          K[32:64] → HMAC 密钥
```

- **先加密后认证** HMAC 覆盖头部、负载与尾部；改算法标识、迭代次数或统计值同样会被发现。
- **常量时间比较** 校验值比较不做短路，避免时序侧信道。
- **参数有上限** 迭代次数限制在 1 ~ 1 000 000，防止伪造文件让还原卡在密钥派生。
- **错误可区分** 口令错误或内容被篡改统一返回 E_AUTH（退出码 3）；客户端重试 3 次后回到编辑态。

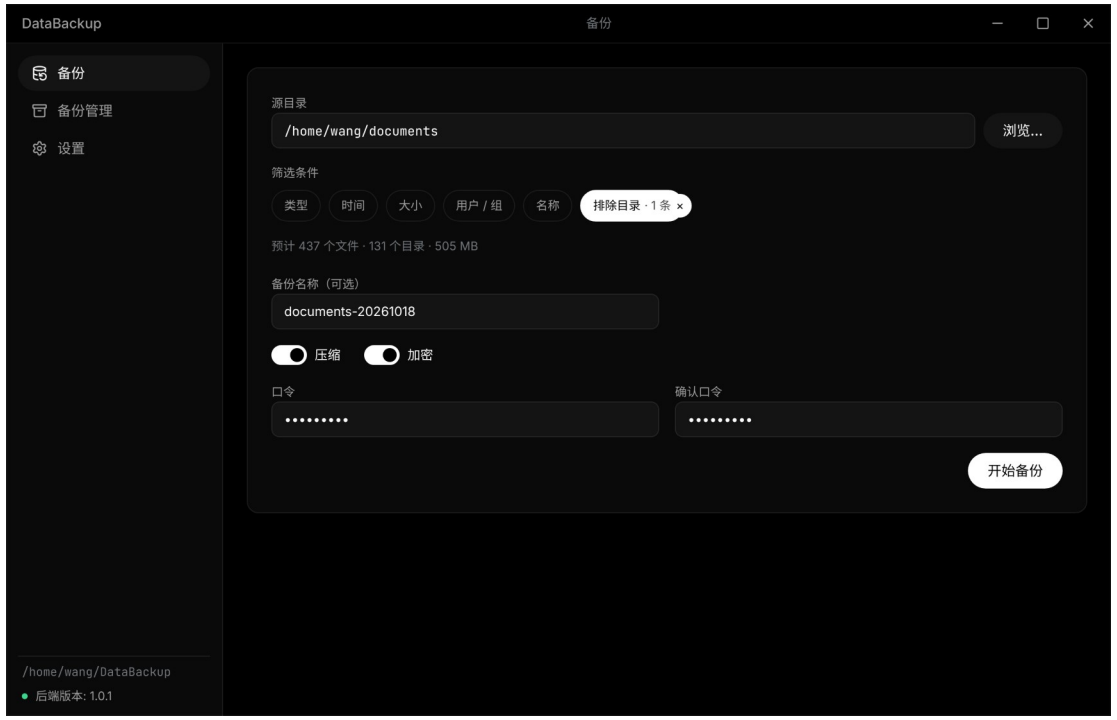
公开向量对照

FIPS-197 （AES 分组）
FIPS 180-4 （SHA-256）
RFC 2104 （HMAC）
RFC 7914 （PBKDF2）
NIST SP 800-38A （CBC）

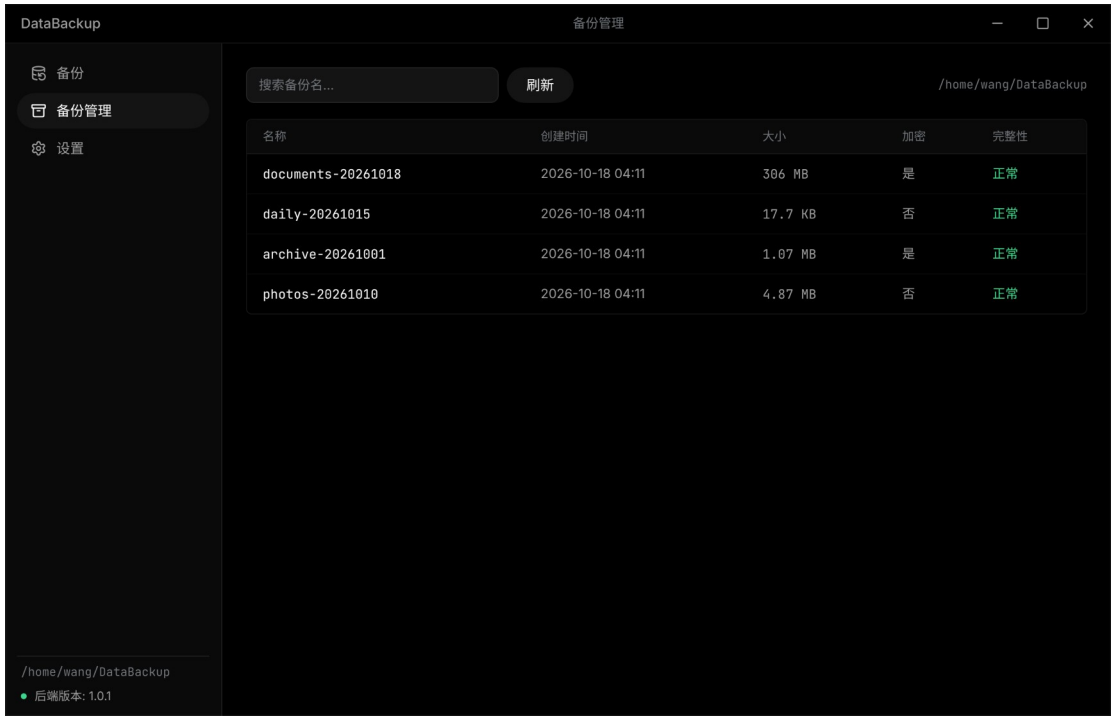
同一组用例在 _test.go 中与标准库输出逐字节对照；生产代码不引用标准库实现。

客户端：三个页面，备份与还原都在一屏内完成

图形界面



备份：源目录、六个筛选维度、压缩与加密、口令与预览



备份管理：列表、完整性标记、详情 Sheet 与还原入口

- Electron 44 + React 19，三页面（备份 / 管理 / 设置）；侧栏折叠、减少动效等偏好可设置。
- 备份任务状态机 9 个状态全部有单元测试覆盖；进度来自后端 JSON Lines，界面只做展示。
- 端到端用 Playwright 驱动打包产物 + 真实后端二进制，覆盖 " 过滤 + 压缩 + 加密 → 还原一致 " 与 E_AUTH 。

工程化：仓库、门禁与协作规则都写进仓库

仓库结构

backend/ Go CLI 与核心库，生产代码零第三方依赖
client/ Electron + React，只做界面与 CLI 调用
docs/ 需求、设计、测试、规范与交付源文件
scripts/ • docker/ 门禁、文档构建与开发镜像

开发方式：编码 agent 按上述规则执行实现与测试，组长负责需求、设计决策、验收与答辩。

质量门禁

检查	覆盖内容	执行时机
make check	格式、 lint 、禁用 import 、构建与单测	每次提交前
make check-all	check + race + 后端 e2e + 客户端 e2e	合并前 / 核心改动
CI	PR 四 job 全绿；打 tag 自动构建与发布	PR 与 Release

协作与约束

- AGENTS.md 是硬约束的唯一来源：零第三方依赖、无学号与凭据、 PDF 只在发布节点生成。
- 任务在 docs/roadmap.md 里按波次与依赖排期；每个 PR 对照任务 ID 与验收标准。
- 行为变更必须同 PR 改文档；提交信息与 Release Notes 有固定写法（ docs/conventions/ ）。

测试：四层用例，需求与用例编号双向可查

层次	范围	手段	数量级
单元测试	各内部包：格式、压缩、加密、过滤、遍历、仓库	go test ， 表驱动 + golden 文件	300+ 用例编号
集成测试	engine 全流水线：七种文件类型、元数据、过滤组合	临时目录树 + 五要素比较器	32 组合矩阵
CLI 黑盒	子命令、参数校验、 JSONL 事件、退出码	testscript （唯一测试依赖）	110 ~ 400 系列
端到端	客户端 + 真实后端二进制	Playwright （含截图回归）	TC-E2E-01 ~ 09

用例编号约定

测试代码里直接写编号（如 // TC-PACK-042 ），需求文档的 FR 编号与测试报告的 TC 编号互相对照，评审时可按编号检索实现与证据。

证据位置

测试报告 §3 覆盖率
§3.3 race 与 fuzz
§4 性能与剖析
§5 健壮性用例表

健壮性单独成组：篡改任一字节、截断、错误口令、取消任务、路径穿越、损坏备份出现在列表中，都有对应用例。

测试结果：核心包覆盖率 87% ~ 100%，竞态与模糊测试干净

包	覆盖率	包	覆盖率
crypto/sha256	100.0%	engine	87.2%
crypto/hmac	100.0%	walk	93.2%
jsonl	100.0%	repo	87.3%
crypto/aes	99.2%	huffman	94.4%
filter	99.0%	pack	95.3%
crypto/pbkdf2	97.1%	container	92.2%

语句覆盖率，2026-10-17 实测；cli 包逻辑经 testscript 子进程执行，不计入包内统计。

其它检查

项目	结果
race 检测	全部通过
FuzzReader	488 万次，无 panic
FuzzDecode	4.06 万次，无 panic
staticcheck / vet	零告警
ESLint / prettier	零告警

过滤 × 压缩 × 加密 组合矩阵

六维过滤各取 " 开 / 关 " 与压缩、加密组合，共 32 组；每组断言备份退出码、容器头标志位与还原后目录树一致。

性能与边界：实测数据与已知限制

场景	数据量	耗时	吞吐
备份（无压缩）	512 MiB	3.22 s	≈167 MB/s
备份（Huffman）	512 MiB	5.51 s	≈97 MB/s
还原（Huffman）	512 MiB	5.67 s	≈95 MB/s
加密容器写入	基准	27.8 ms/op	≈37.8 MB/s
10,000 个小文件	260 KB	0.16 s	—

WSL2 Ubuntu 24.04 + NVMe，真实二进制；微基准为 go test -bench， benchtime=2s。

pprof 热点（加密容器写入）

函数	占比
aes.addRoundKey	18.6%
aes.shiftRows	18.3%
aes.mixColumns	15.6%
sha256.block	15.6%
aes.subBytes	12.2%

采样 3 s； WSL2 不提供 perf_event，用 Go pprof 替代。

已知限制

- 超过 10 万个条目的目录树没有做过端到端实测；测试报告 §6.3 对此有说明。
- Huffman 为单线程实现，压缩吞吐低于不压缩路径；加密写入约 38 MB/s，是当前最慢环节。
- 仓库是普通目录：没有增量备份、去重与并发写保护，同一仓库并发备份只能靠 E_EXISTS 兜底。
- Windows 便携包会静默跳过 uid/gid、权限位、设备节点与硬链接身份，仅作演示用途。

交付物：一次构建产出命令行、两种安装包与全部文档

产物	大小	说明
DataBackup-1.0.1.AppImage	139.8 MB	单文件免安装，内置后端二进制
databackup-client_1.0.1_amd64.deb	107.6 MB	dpkg 安装，菜单入口 /opt/DataBackup
databackup-1.0.1-linux-x86_64.tar.gz	1.6 MB	独立 CLI，可脚本化调用
databackup-1.0.1-source.tar.gz	6.0 MB	源码包，含构建脚本
文档 PDF × 6	3.8 MB	需求、设计、测试、实验报告、 PPT 、进度

v1.0.1 Release 资产，同时提供 sha256sums.txt ；打 tag 由 CI 自动构建并发布。

运行

```
sudo dpkg -i databackup-client_1.0.1_amd64.deb
databackup backup --source ~/docs --repo ~/backups \
          --compress huffman --encrypt aes256 --password-stdin
```

目标平台 Ubuntu 22.04 / 24.04 。 WSL2 + WSLg 的安装、依赖与已知问题见 docs/client/wsl-guide.md 。

演示视频 ≤ 3 分钟，按 docs/deliverables/06- 演示视频脚本 .md 录制。

小结：课程要求的功能全部落地，边界清楚

已完成

- 基础备份 / 还原 + 全部扩展项：七类文件对象、完整元数据、六维过滤、自研格式 / 压缩 / 加密、 Electron 客户端。
- 自研实现有公开向量与 golden 文件对照；客户端走真实后端端到端验证。
- 从需求编号到测试编号、从 Makefile 到 CI 与 Release，工程链路完整可复现。

未做与后续

- 增量备份、去重、并发写保护：都属于仓库层设计，需要先定新的格式版本。
- 更大规模目录树的实测（10 万条目以上）与多线程压缩。
- Windows 版的元数据能力受平台限制，短期只作为演示兜底。

代码规模：后端 Go 生产 6,755 行、测试 7,157 行；客户端 TypeScript/TSX 4,749 行、测试 2,325 行；文档 Markdown 3,600 余行（不含图）。

谢谢

演示见视频或现场：过滤 + 压缩 + 加密备份 → 口令还原 → 目录树比对。